Birla Vishvakarma Mahavidyalaya Engineering College
Vallabh Vidyanagar, Gujarat

Department of Electronics Engineering

A Project Report on

ROS 2 Based Digital Twin for Robotic Arm Simulation and Control

Submitted in partial fulfillment of the requirements for the course

3EL13 – Electronics System Design Laboratory

Submitted by:
Dhruv Thakur (23EL020)

Under the guidance of:
Dr. Dipakkumar M Patel

Semester VI – Academic Year 2025-26

May 2026

Certificate

This is to certify that the project report entitled "ROS 2 Based Digital Twin for Robotic Arm Simulation and Control" has been carried out by Dhruv Thakur (23EL020) and Aaditya Popli (23EL023) under my guidance in partial fulfillment of the requirements for the course 3EL13 – Electronics System Design Laboratory during Semester VI of the academic year 2025-26 at the Department of Electronics Engineering, Birla Vishvakarma Mahavidyalaya Engineering College, Vallabh Vidyanagar.

Date: May 2026
Place: Vallabh Vidyanagar

Dr. Dipakkumar M Patel
Project Guide

Acknowledgement

We would like to express our sincere gratitude to our project guide Dr. Dipakkumar M Patel for his invaluable guidance, encouragement, and support throughout the development of this project. His expertise in electronics system design and his constructive suggestions was instrumental in shaping this work.

We are grateful to the Department of Electronics Engineering, Birla Vishvakarma Mahavidyalaya Engineering College, for providing us with the necessary resources and laboratory facilities to carry out this project.

We extend our thanks to our classmates and peers who provided helpful feedback and discussions during the course of this work.

Dhruv Thakur (23EL020)
Aaditya Popli (23EL023)

Abstract

This project presents the design and implementation of a Digital Twin system for a 6-degree-of-freedom (6-DOF) robotic arm using the Robot Operating System 2 (ROS 2) Jazzy framework, Gazebo Harmonic simulator, and MoveIt 2 motion planning library. A digital twin is a virtual replica of a physical system that mirrors its real-time behavior, enabling simulation, testing, and validation before deployment on actual hardware.

The system establishes bidirectional communication between a simulated robotic arm in Gazebo and a physical 6-DOF aluminium robotic arm controlled by an Arduino Uno through a PCA9685 16-channel servo driver. Joint angles computed by MoveIt 2's motion planner are extracted from the /joint_states ROS 2 topic, converted from radians to servo-compatible degrees, and transmitted via USB serial communication to the Arduino, which drives six MG995 servo motors through the PCA9685 I2C interface.

Key features of the system include sequential servo homing for safe startup, configurable per-joint direction reversal and angle limits, real-time synchronization between the simulated and physical arms, and a modular Python-based ROS 2 bridge node that translates between the simulation framework and the embedded hardware. The system was developed and tested in Ubuntu 24.04 LTS running inside Oracle VirtualBox, demonstrating that a complete robotics development environment can be established even without dedicated hardware.

Keywords: *Digital Twin, ROS 2, Gazebo, MoveIt 2, Robotic Arm, 6-DOF, Arduino, PCA9685, Servo Control, Motion Planning, Serial Communication*

Table of Contents

List of Figures

List of Tables

Chapter 1: Introduction

1.1 Background and Motivation

The field of robotics has seen rapid advancements in recent years, with robotic arms being employed in manufacturing, healthcare, agriculture, and research applications. A significant challenge in robotics development is the gap between simulation and physical hardware — motions that work perfectly in simulation may fail on real hardware due to mechanical constraints, servo limitations, or timing issues.

The concept of a Digital Twin addresses this challenge by creating a virtual replica of a physical system that operates in real-time synchronization. In the context of robotic arms, a digital twin allows developers to plan, test, and validate robot motions in simulation before executing them on physical hardware, thereby reducing the risk of damage and accelerating the development cycle.

The Robot Operating System 2 (ROS 2) has emerged as the de facto standard framework for robotics software development, providing a modular, distributed architecture for robot control. Combined with the Gazebo physics simulator and MoveIt 2 motion planning library, ROS 2 provides a complete ecosystem for developing sophisticated robotic applications.

1.2 Problem Statement

To design and implement a digital twin system that synchronizes a simulated 6-DOF robotic arm (running in Gazebo with MoveIt 2 motion planning) with a physical robotic arm, such that motions planned in the simulation are executed simultaneously on the real hardware through serial communication and servo control.

1.3 Objectives

1. Set up a complete ROS 2 Jazzy development environment with Gazebo Harmonic simulator and MoveIt 2 motion planning framework.
2. Create a simulated 6-DOF robotic arm model using URDF/Xacro with appropriate joint limits, inertial properties, and visual meshes.
3. Configure MoveIt 2 for motion planning with multiple planning algorithms (OMPL, Pilz, STOMP).
4. Design and implement a serial communication bridge between ROS 2 and Arduino for real-time joint angle transmission.
5. Develop Arduino firmware for controlling six servo motors through a PCA9685 16-channel PWM driver via I2C.
6. Achieve real-time synchronization between the simulated and physical robotic arms.
7. Implement safety features including sequential homing, joint angle clamping, and graceful shutdown.

1.4 Scope of the Project

This project covers the complete pipeline from simulation to hardware control: ROS 2 environment setup, Gazebo simulation, MoveIt 2 motion planning, serial communication protocol design, Arduino firmware development, and PCA9685 servo driver integration. The project demonstrates the digital twin concept using pre-programmed and interactively planned motions. Vision-based object detection and autonomous pick-and-place are identified as future extensions.

Chapter 2: System Architecture and Design

2.1 Overall System Architecture

The digital twin system consists of two parallel paths that operate in synchronization: a simulation path and a hardware path. Both paths originate from the same MoveIt 2 motion planner, ensuring identical joint angle commands reach both the simulated and physical robots.

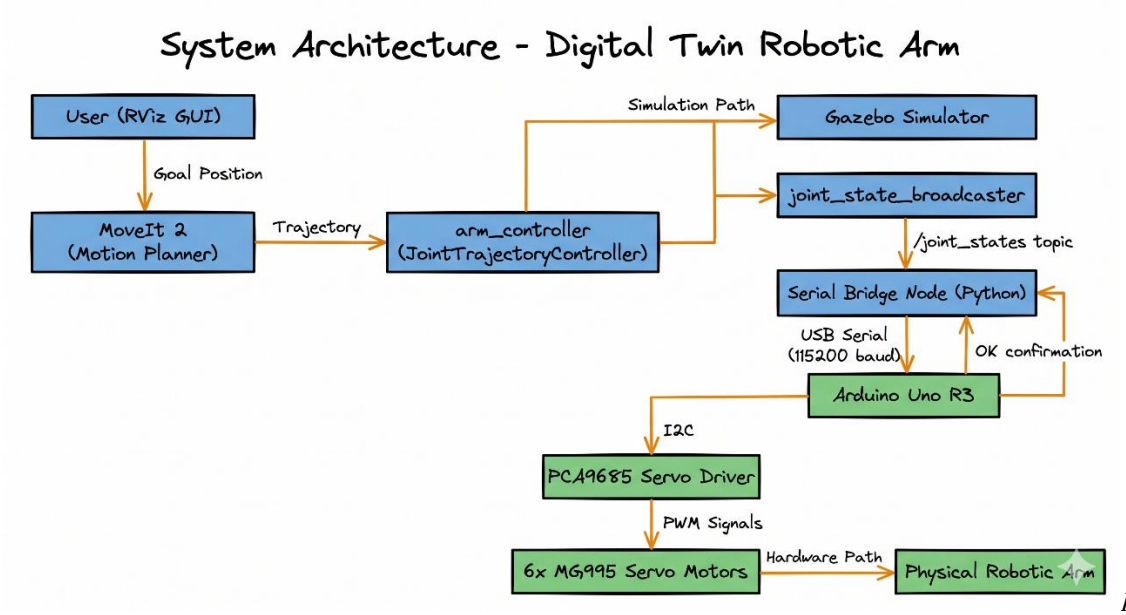


Figure 2.1: Overall System Architecture

The data flow proceeds as follows:

8. The user sets a desired end-effector position using the MoveIt 2 interactive marker in RViz.
9. MoveIt 2 computes the inverse kinematics solution and generates a collision-free trajectory using the configured planning algorithm.
10. The trajectory is sent to the JointTrajectoryController, which executes it in the Gazebo simulation.
11. The joint_state_broadcaster publishes the current joint angles to the /joint_states ROS 2 topic.
12. The Python serial bridge node subscribes to /joint_states, extracts the six joint angles, converts them from radians to servo degrees, and transmits them via USB serial to the Arduino.
13. The Arduino firmware parses the received angles and sends I2C commands to the PCA9685, which generates precise PWM signals for each servo motor.

2.2 Software Architecture

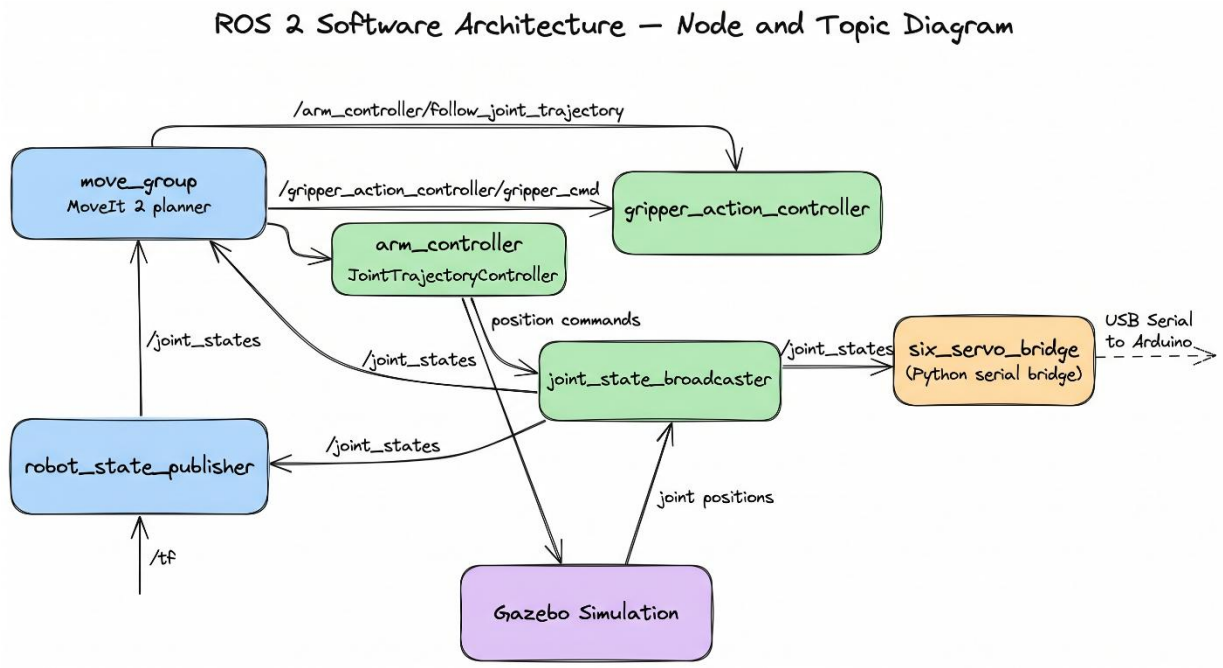


Figure 2.2: Software Architecture

The software stack consists of multiple ROS 2 nodes communicating through topics and action servers:

- **move_group:** The central MoveIt 2 node that handles motion planning, inverse kinematics, and trajectory generation.
- **robot_state_publisher:** Broadcasts the robot's TF (Transform) tree based on the URDF model and current joint states.
- **joint_state_broadcaster:** Reads joint positions from the hardware interface and publishes them to /joint_states.
- **arm_controller:** A JointTrajectoryController that interpolates trajectory waypoints and sends position commands to the simulated hardware at 100 Hz.
- **gripper_action_controller:** Controls the gripper open/close actions via the GripperCommand action interface.
- **six_servo_bridge:** Custom Python node that bridges ROS 2 and Arduino via serial communication.

2.3 Hardware Architecture

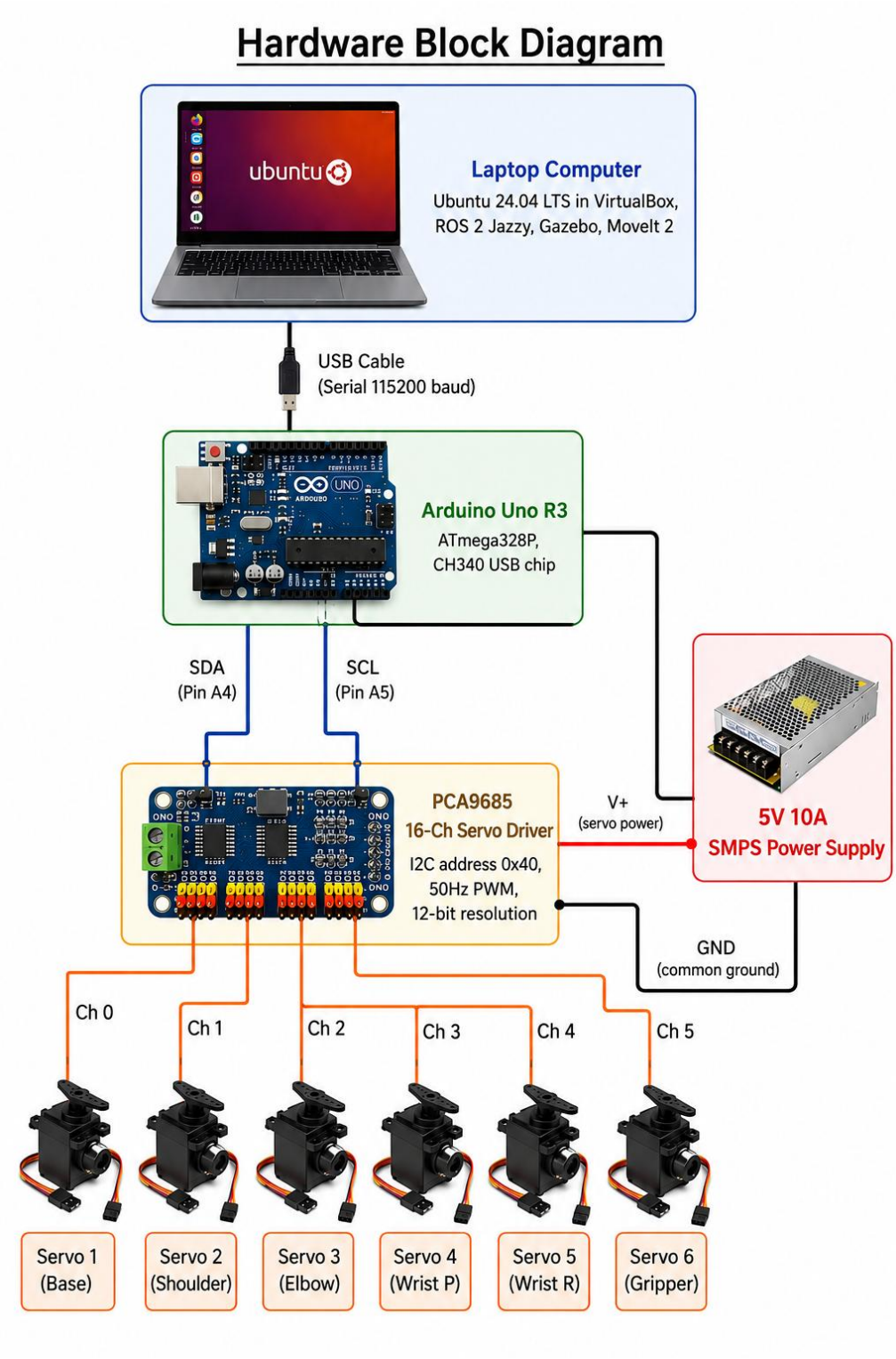


Figure 2.3: Hardware Block Diagram

The hardware consists of a laptop computer running Ubuntu 24.04 LTS in VirtualBox (hosting ROS 2, Gazebo, and MoveIt 2), connected via USB serial to an Arduino Uno R3. The Arduino communicates with a PCA9685 16-channel servo driver over I2C, which generates stable 50 Hz PWM signals for six MG995 servo motors. An external 5V 10A SMPS power supply provides adequate current for all six servos through the PCA9685 V+ terminal.

2.4 Communication Protocol Design

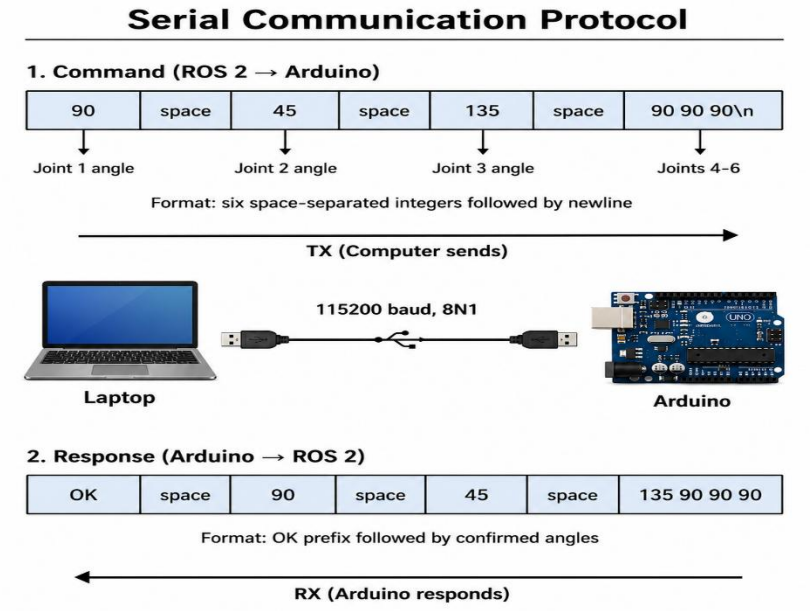


Figure 2.4: Serial Communication Protocol

A lightweight ASCII-based serial protocol was designed for communication between the ROS 2 bridge node and the Arduino firmware:

Command format (ROS 2 → Arduino): Six space-separated integer angles followed by a newline character.

Example: *"90 45 135 90 90 90\n"*

Response format (Arduino → ROS 2): "OK" followed by six space-separated confirmed angles.

Example: *"OK 90 45 135 90 90 90"*

This protocol was chosen for its simplicity, debuggability (human-readable in serial monitors), and reliability. String-based parsing was used instead of Arduino's Serial.parseInt() function, which was found to misread values due to timeout issues during development.

Joint	MoveIt Name	PCA9685 Ch	Servo	Direction	Range
1 (Base)	link1_to_link2	0	MG995	Normal	10°-170°
2 (Shoulder)	link2_to_link3	1	MG995	Reversed	30°-150°
3 (Elbow)	link3_to_link4	2	MG995	Normal	40°-140°
4 (Wrist P)	link4_to_link5	3	MG995	Normal	40°-140°
5 (Wrist R)	link5_to_link6	4	MG995	Normal	20°-160°
6 (Gripper)	link6_to_link6_flange	5	MG995	Normal	30°-150°

Table 2.1: Joint Mapping – MoveIt Joint Names to Servo Channels

Chapter 3: Software Implementation

3.1 ROS 2 Jazzy Setup

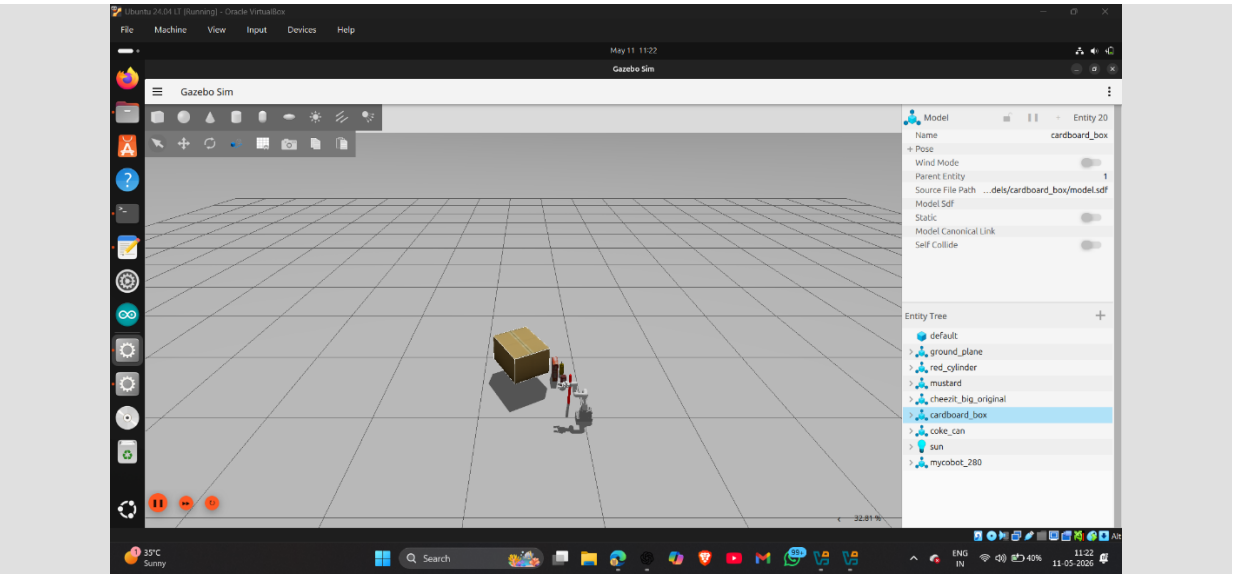
The development environment was established on Ubuntu 24.04 LTS running inside Oracle VirtualBox 7.0 on a Windows host machine. ROS 2 Jazzy Jalisco (the latest LTS release) was installed along with the following key packages:

- ros-jazzy-desktop — Core ROS 2 with RViz visualization
- ros-jazzy-ros-gz — Gazebo Harmonic integration
- ros-jazzy-ros2-control and ros-jazzy-ros2-controllers — Robot control framework
- ros-jazzy-gz-ros2-control — Gazebo-ROS 2 Control bridge
- ros-jazzy-moveit — MoveIt 2 motion planning framework
- ros-jazzy-xacro — XML macro processor for URDF files

The ROS 2 workspace was created at ~/ros2_ws/ and the mycobot_ros2 package collection was cloned from the Automatic Addison GitHub repository (branch: jazzy). Seven packages were successfully built using the colcon build system.

Due to VirtualBox's limited GPU support, software rendering was enabled for Gazebo using the environment variables LIBGL_ALWAYS_SOFTWARE=1 and MESA_GL_VERSION_OVERRIDE=3.3.

3.2 Gazebo Simulation Environment



The Gazebo Harmonic physics simulator was used to create a virtual environment containing the robotic arm and interaction objects (red cylinder, mustard bottle, cheezit box, cardboard box, and coke can). The simulation runs at real-time speed with the DART physics engine, providing accurate rigid body dynamics, collision detection, and gravity simulation at -9.8 m/s^2.

The gz_ros2_control plugin bridges Gazebo and ROS 2 Control, allowing the same controller configuration to work in both simulation and on real hardware. Three controllers are loaded: joint_state_broadcaster (publishes joint positions), arm_controller (JointTrajectoryController at 100 Hz), and gripper_action_controller (GripperActionController).

3.3 MoveIt 2 Configuration

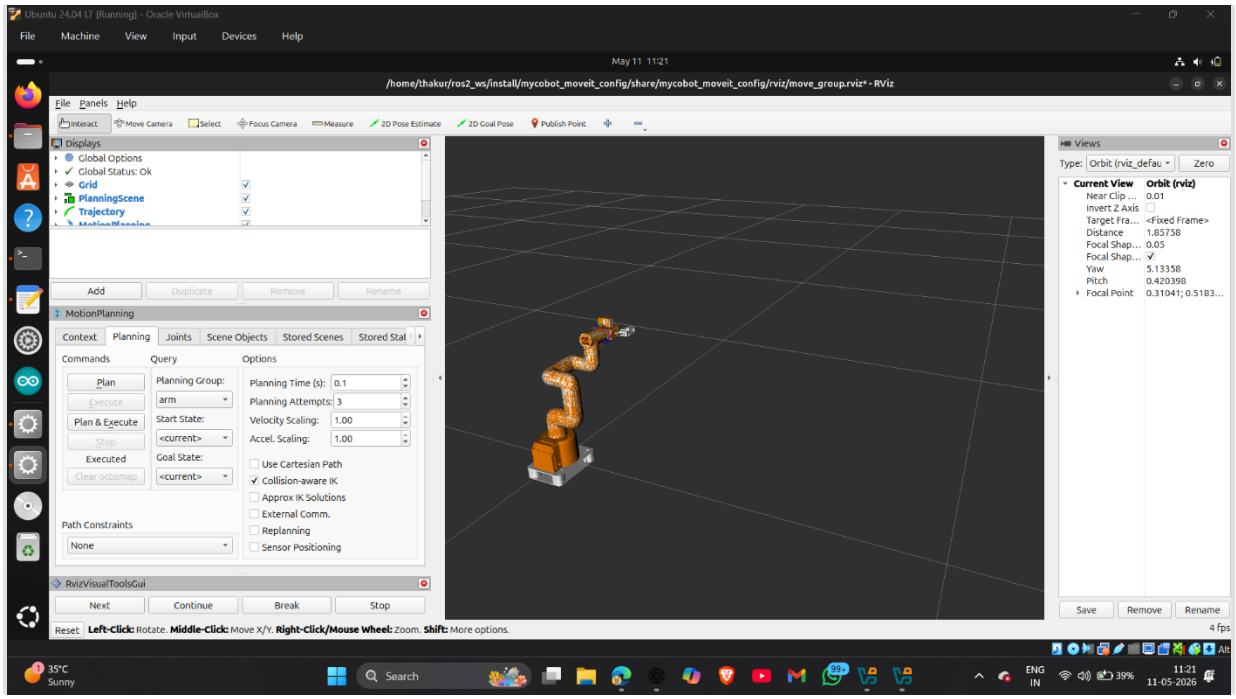


Figure 3.2: MoveIt 2 Motion Planning in RViz — Screenshot showing the orange goal state, planned trajectory, and Motion Planning panel

MoveIt 2 was configured for the robotic arm with the following components:

- **SRDF (mycobot_280.srdf):** Defines three planning groups (arm, gripper, arm_with_gripper), named poses (home), and the self-collision matrix.
- **kinematics.yaml:** Configures the KDL (Kinematics and Dynamics Library) inverse kinematics solver with a 0.05-second timeout and 5 retry attempts.
- **moveit_controllers.yaml:** Maps the arm_controller (FollowJointTrajectory action) and gripper_action_controller (GripperCommand action) for trajectory execution.
- **joint_limits.yaml:** Defines velocity and acceleration limits for safe motion planning.

Three motion planning algorithms were configured: OMPL (sampling-based, RRTConnect), Pilz Industrial Motion Planner (PTP, LIN, CIRC), and STOMP (trajectory optimization).

3.4 ROS 2 Serial Bridge Node

A Python-based ROS 2 node (six_servo_bridge.py) was developed as the interface between the simulation framework and the physical hardware. The node performs the following functions:

14. **Subscribes to /joint_states topic** to receive real-time joint angle updates.
15. **Extracts the six arm joint angles** by name matching (link1_to_link2 through link6_to_link6_flange).
16. **Converts radians to servo degrees:** servo_angle = degrees(radians) × direction + offset, clamped to [min, max].
17. **Sends the six angles** as a space-separated string via USB serial at 115200 baud.
18. **Reads confirmation** from Arduino to verify command reception.

Safety features implemented in the bridge node:

- **Sequential homing:** On startup, each servo is moved to its home position (90°) one at a time with a 1-second delay, preventing current spikes and mechanical jerks.
- **Graceful shutdown:** On Ctrl+C, all servos return to home position sequentially before the program exits.
- **Per-joint configuration:** Each joint has independently configurable direction, offset, minimum angle, and maximum angle.
- **Change detection:** Commands are sent to Arduino only when angles change, preventing unnecessary serial traffic.

3.5 Arduino Firmware

Custom firmware was developed for the Arduino Uno R3 using the Arduino IDE 1.8.19. The firmware uses the Adafruit PWM Servo Driver library to communicate with the PCA9685 via I2C. Key design decisions:

- **String-based serial parsing:** Characters are accumulated into a String until a newline is received, then parsed with toInt(). This replaced Serial.parseInt() which was found to misread values due to its 1-second timeout.
- **Constrained angles:** All received angles are clamped to 10°-170° as a hardware safety measure.
- **Confirmation response:** After moving servos, Arduino sends "OK" with the confirmed angles, enabling the bridge node to verify command execution.

Chapter 4: Hardware Implementation

4.1 Components Used

Sr.	Component	Specification	Qty	Cost (₹)	Purpose
1	6-DOF Robotic Arm Kit (RKI-2533)	Aluminium frame, assembled	1	14,990	Mechanical structure
2	Arduino Uno R3 (CH340)	ATmega328P, USB-Serial	1	500	Microcontroller
3	PCA9685 Servo Driver	16-ch, I2C, 12-bit PWM	1	250	Servo PWM generation
4	MG995 Servo Motor	11 kg·cm, 4.8-7.2V, Metal gear	6	1,194	Joint actuation
5	5V 10A SMPS Power Supply	AC-DC, 50W output	1	450	Servo power
6	Jumper Wires	M-M, M-F assorted	1 set	150	Connections
7	USB-B Cable	USB-A to USB-B, 1m	1	80	Arduino-PC serial

Table 4.1: Bill of Materials

4.2 Wiring and Connections

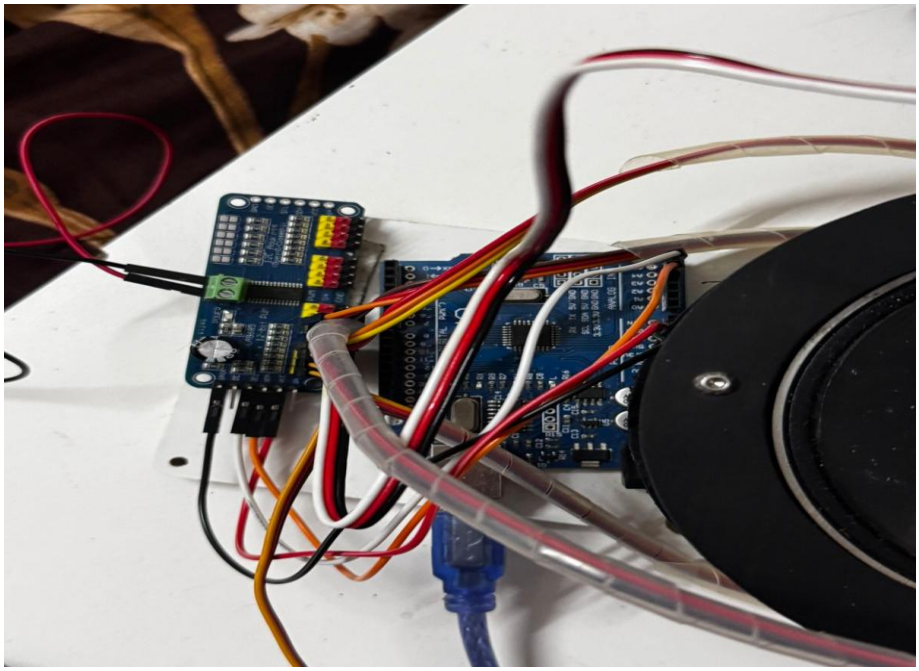


Figure 4.1: PCA9685 Servo Driver Wiring Diagram — showing Arduino Uno connected to PCA9685 via I2C (SDA=A4, SCL=A5), 6 servos on channels 0-5, external PSU on V+ terminal

Arduino Uno → PCA9685 (I2C):

- Arduino A4 (SDA) → PCA9685 SDA
- Arduino A5 (SCL) → PCA9685 SCL
- Arduino 5V → PCA9685 VCC (logic power)
- Arduino GND → PCA9685 GND

External 5V 10A PSU → PCA9685 (servo power):

- PSU (+) → PCA9685 V+ screw terminal
- PSU (-) → PCA9685 GND screw terminal

PCA9685 → Servo Motors:

- Channel 0: Joint 1 (Base rotation)
- Channel 1: Joint 2 (Shoulder)
- Channel 2: Joint 3 (Elbow)
- Channel 3: Joint 4 (Wrist pitch)
- Channel 4: Joint 5 (Wrist roll)
- Channel 5: Joint 6 (Gripper/Flange)

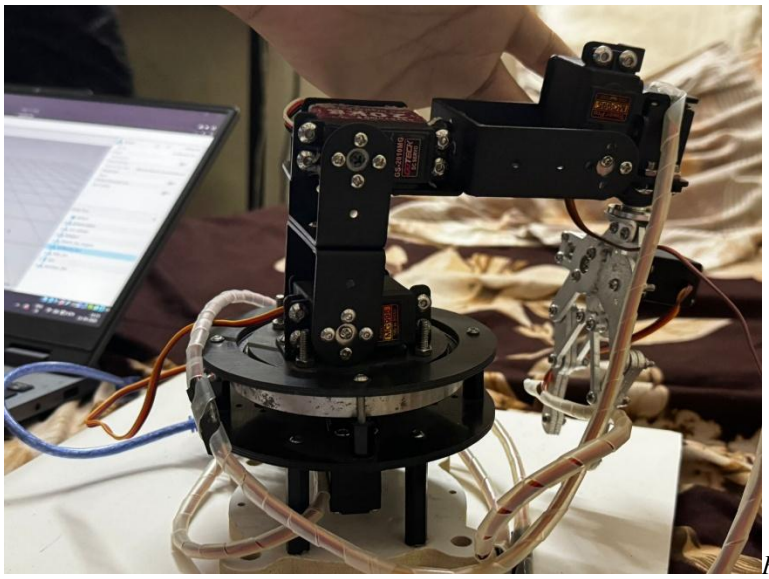


Figure 4.3: Complete Hardware Setup — Photo showing the assembled arm with PCA9685, Arduino, and power supply

4.3 Servo Motor Configuration

Joint	Offset	Direction	Min	Max	Home	Reason for Limits

Base	90°	Normal (+1)	10°	170°	90°	Full rotation safe
Shoulder	90°	Reversed (-1)	30°	150°	90°	Reversed to match simulation
Elbow	90°	Normal (+1)	40°	140°	90°	Restricted: was overshooting
Wrist P	90°	Normal (+1)	40°	140°	90°	Restricted: was overshooting
Wrist R	90°	Normal (+1)	20°	160°	90°	Slight restriction
Gripper	90°	Normal (+1)	30°	150°	90°	Prevents over-gripping

Table 4.2: Servo Configuration Parameters

4.4 Power Supply Design

Each MG995 servo motor draws 500 mA to 2 A under load. With six servos operating simultaneously, the maximum current draw is approximately 12 A. A 5V 10A SMPS was selected to provide adequate current with headroom. The power supply connects exclusively to the PCA9685 V+ terminal; servos are never powered from the Arduino's 5V pin, which can supply only 500 mA total.

A common ground connection between the Arduino, PCA9685, and external power supply ensures proper PWM signal reference levels.

Chapter 5: Results and Testing

5.1 Simulation Results

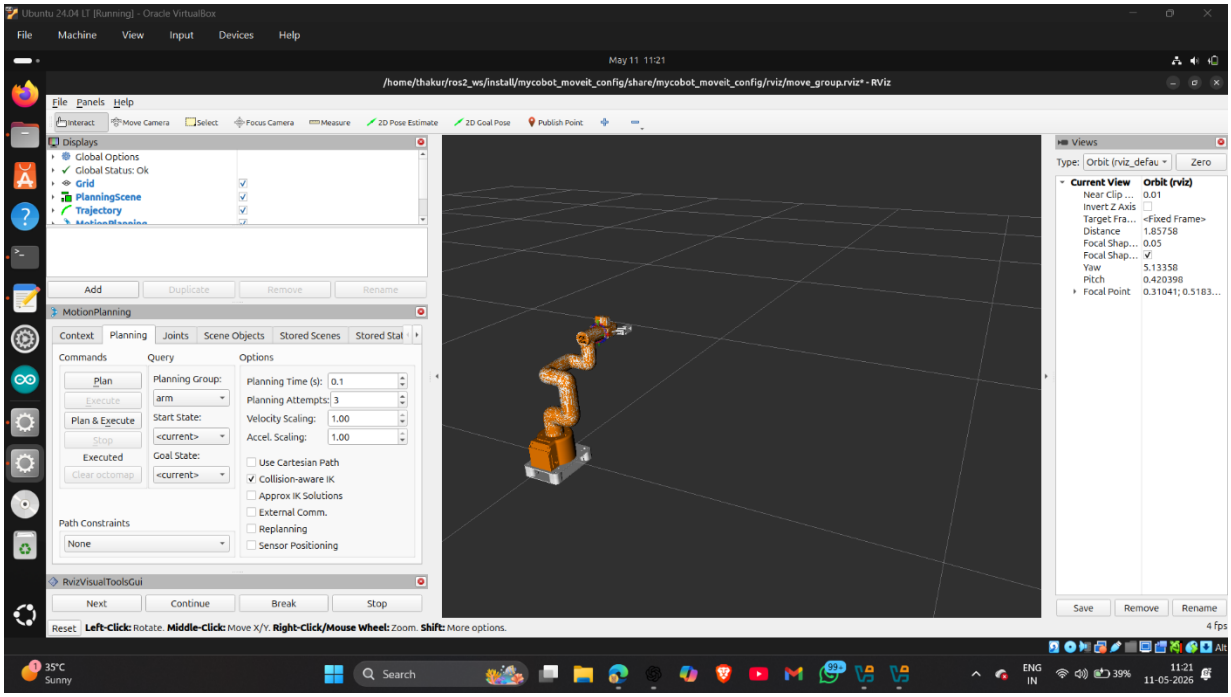


Figure 5.1: RViz Interactive Marker for Goal Setting — Screenshot of RViz showing the interactive marker on the arm with the Motion Planning panel

The simulation was successfully established with all three controllers (joint_state_broadcaster, arm_controller, gripper_action_controller) in active state. Motion planning using OMPL RRTConnect, Pilz PTP, and Pilz LIN planners was tested and verified. The simulated arm responded correctly to all planned trajectories, with real-time visualization in both Gazebo and RViz.

5.2 Serial Communication Verification

Serial communication between the ROS 2 bridge node and Arduino was verified in three stages:

19. **Arduino standalone test:** Commands sent via Serial Monitor were correctly parsed and servos responded. For example, sending "90 45 135 90 90 90" correctly moved Joint 2 to 45° and Joint 3 to 135°.
20. **Python serial test:** A Python script using the pyserial library confirmed bidirectional communication. The Arduino responded with correct "OK" confirmations.
21. **ROS 2 topic-based test:** Fake joint states published via ros2 topic pub were correctly received by the bridge node, converted to servo degrees, and transmitted to Arduino.

5.3 Single Servo Test

Initial testing was performed with a single SG90 servo motor connected to PCA9685 Channel 0, representing the base joint. The following conversion was verified:

MoveIt sends 0.0 radians → Bridge converts to 90° → Servo centers correctly.

MoveIt sends +0.785 radians (+45°) → Bridge converts to 135° → Servo rotates correctly.

MoveIt sends -1.1028 radians (-63°) → Bridge converts to 27° → Servo rotates correctly.

5.4 Full 6-DOF Digital Twin Test

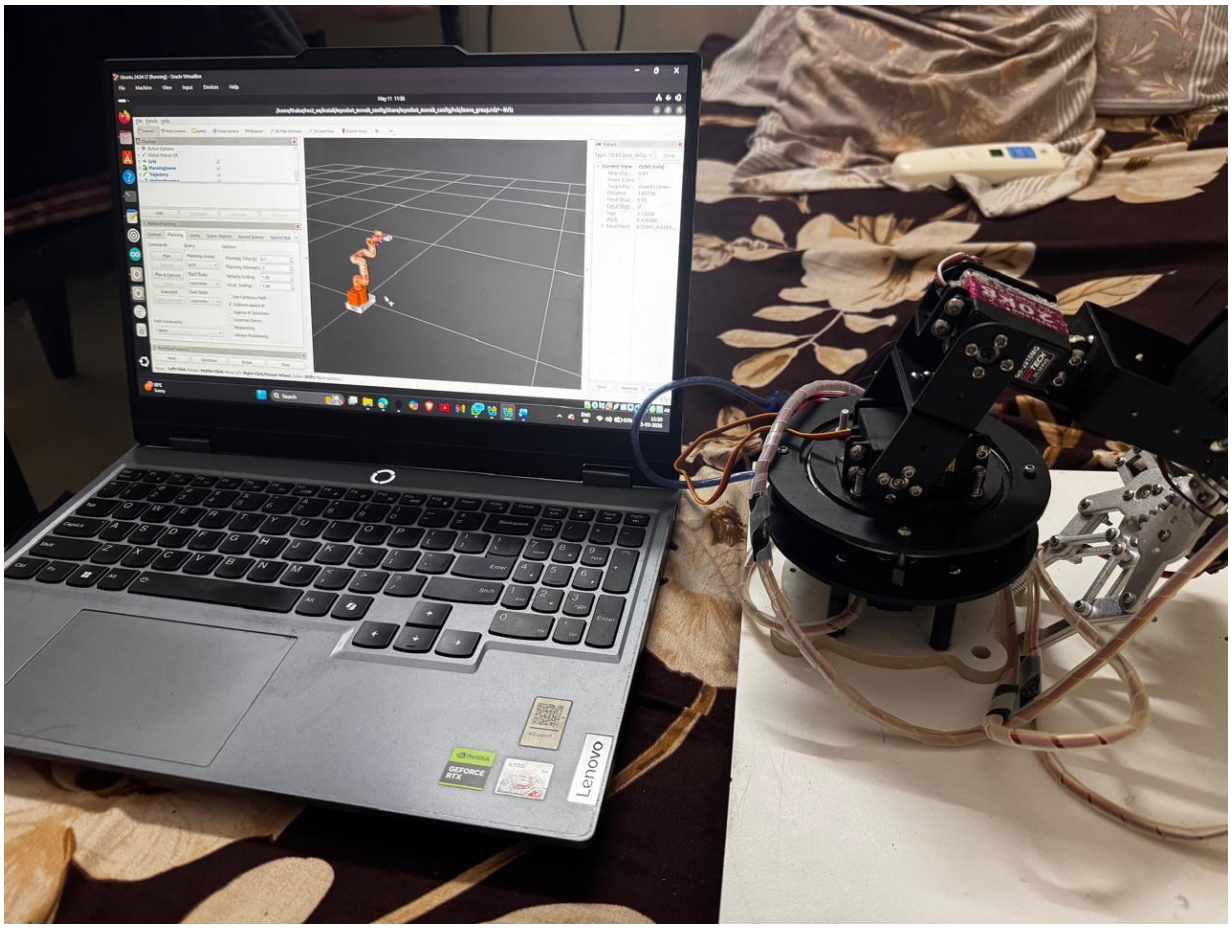


Figure 5.4: Digital Twin in Action — Side-by-side photo of the RViz/Gazebo simulation and the physical arm in the same pose

The complete digital twin was tested by planning various motions in MoveIt 2 and verifying that the physical arm tracked the simulation:

Test	MoveIt Command	Physical Arm Response	Result
Home position	All joints 0 rad	All servos at 90°	PASS
Base rotation	Joint 1: +0.785 rad	Base servo: 135°	PASS
Shoulder + Elbow	J2: -0.5, J3: 0.3 rad	Correct servo angles	PASS
Full arm motion	6-joint trajectory	All 6 servos follow	PASS
Sequential homing	Bridge startup	Joints home one by one	PASS
Safe shutdown	Ctrl+C	All joints return home	PASS

Table 5.1: Test Results Summary

5.5 Problems Encountered and Solutions

Problem	Cause	Solution
Gazebo black screen in VM	VirtualBox GPU doesn't support OpenGL	LIBGL_ALWAYS_SOFTWARE=1 and MESA_GL_VERSION_OVERRIDE=3.3
Controller activation timeout	VM too slow for default timing	Launch Gazebo first, wait 60s, then launch MoveIt separately
Arduino upload fails (stk500 error)	Old sketch holds serial port	Unplug USB, click Upload, quickly replug USB
Serial.parseInt() misreads values	1-second timeout in parseInt()	String-based char-by-char parsing with toInt()
Shoulder servo moves backwards	Physical servo mounted in reverse	Set direction=-1 in bridge config
Elbow/wrist servos overshoot	Full 0-180° range exceeds mechanical limits	Restricted to 40°-140° safe range
Startup jerk damages servo	All 6 servos moving simultaneously	Sequential homing: one joint at a time with 1s delay

Table 5.2: Problems Encountered and Solutions

Chapter 6: Conclusion and Future Scope

6.1 Conclusion

This project successfully demonstrates the implementation of a digital twin system for a 6-DOF robotic arm using ROS 2 Jazzy, Gazebo Harmonic, and MoveIt 2. The system achieves real-time synchronization between a simulated robotic arm and physical hardware through a well-designed serial communication bridge.

The key accomplishments of this project are:

22. A complete ROS 2 development environment was established from scratch, including Gazebo physics simulation and MoveIt 2 motion planning, running on Ubuntu 24.04 inside VirtualBox.
23. A modular Python-based serial bridge was developed that translates MoveIt 2 joint angle commands (in radians) to PCA9685 servo driver commands (in degrees) with per-joint calibration support.
24. Safety features including sequential servo homing, configurable joint limits, direction reversal, and graceful shutdown were implemented to protect the hardware.
25. The digital twin concept was validated — motions planned interactively in MoveIt 2 are executed identically on both the simulated and physical robotic arms.

The project demonstrates that a fully functional robotics development platform can be established with minimal hardware investment, using open-source tools (ROS 2, Gazebo, MoveIt 2) and affordable components (Arduino Uno, PCA9685, MG995 servos).

6.2 Future Scope

26. **Custom URDF for exact arm dimensions:** Creating a URDF that matches the physical arm's link lengths will improve position accuracy of the digital twin.
27. **Vision-based pick and place:** Adding a camera (Raspberry Pi Camera Module or USB webcam) with OpenCV-based object detection to enable autonomous pick-and-place operations.
28. **Gesture control:** Integrating MediaPipe hand tracking to allow the arm to be controlled by hand gestures via a webcam.
29. **Raspberry Pi integration:** Moving ROS 2 from the laptop to a Raspberry Pi 4 for a standalone, portable system.
30. **STM32 with micro-ROS:** Replacing Arduino with an STM32 Nucleo board running micro-ROS for native ROS 2 integration at the embedded level.
31. **Force/torque feedback:** Adding current sensors on servo lines to detect load and provide force feedback to the motion planner.

References

[1] Open Robotics, "ROS 2 Documentation – Jazzy Jalisco," docs.ros.org, 2024. Available: https://docs.ros.org/en/jazzy/

[2] Open Robotics, "Gazebo Sim Documentation," gazebosim.org, 2024. Available: https://gazebosim.org/docs

[3] PickNik Robotics, "MoveIt 2 Documentation," moveit.picknik.ai, 2024. Available: https://moveit.picknik.ai/main/

[4] A. Sears-Collins, "How to Simulate a Robotic Arm in Gazebo – ROS 2 Jazzy," Automatic Addison, 2024. Available: https://automaticaddison.com/

[5] Adafruit Industries, "PCA9685 16-Channel PWM/Servo Driver," Adafruit Learning System, 2023. Available: https://learn.adafruit.com/16-channel-pwm-servo-driver

[6] Robokits India, "Robotic Arm 6 DOF Aluminium Clamp DIY (RKI-2533)," robokits.co.in, 2024. Available: https://robokits.co.in/

[7] K. M. Lynch and F. C. Park, "Modern Robotics: Mechanics, Planning, and Control," Cambridge University Press, 2017. Available: http://modernrobotics.org/

[8] A. Sodemann, "Robogrok Robotics Course," robogrok.com, 2024. Available: https://www.robogrok.com/

[9] Arduino, "Arduino Uno R3 Documentation," arduino.cc, 2024. Available: https://docs.arduino.cc/hardware/uno-rev3/

[10] TowerPro, "MG995 Metal Gear Servo Motor Datasheet," towerpro.com.tw, 2023.